Design and Implementation of a Multi-Threaded HTTP/1.1 Web Server from Scratch Using Raw TCP Sockets

Bounded pthread worker-pool concurrency, RFC 9112 byte-stream framing, and Queue-and-Latency-Aware Admission Control (QLAAC) in ISO C17 on POSIX sockets

Yash Pradhan — 24BCE0702

Programme: B.Tech. in Computer Science and Engineering
School: School of Computer Science and Engineering (SCOPE), VIT
Course: Innovative Evaluation Practices for BCSE308P — Computer Networks Lab

Anjini Pandey — 24BCE0714

Programme: B.Tech. in Computer Science and Engineering
School: School of Computer Science and Engineering (SCOPE), VIT
Semester: Fall Semester 2026–27 · Final Project Submission & Evaluation

ISO C17 & POSIX.1-2008 · 4 Worker Pthreads & 64-Slot Bounded FIFO Queue · Same Executable Baseline vs. QLAAC (-a) · 80 Historical v4 + 80 Fresh Rerun Trials · 92 Automated Tests

Problem & Motivation: Queueing Delay Dominates Tail Latency

Why bounded thread-pool servers experience severe connection-queue wait and p99 inflation when clients exceed workers

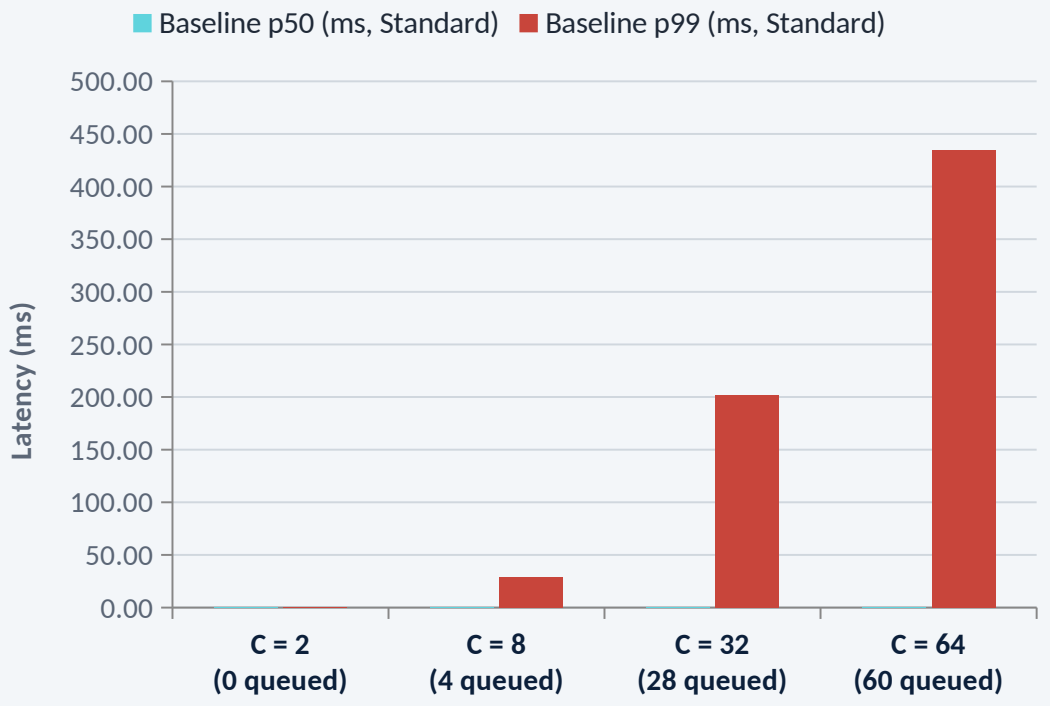
Finite Worker Pool vs. Persistent Keep-Alive Clients

- **TCP Byte Stream & Worker Pinning:** In a synchronous thread-pool server, each accepted persistent HTTP/1.1 connection occupies one worker thread while reading, serving, or waiting between keep-alive requests.
- **Fixed Resource Limits (W = 4, Q = 64):** To bound memory and file descriptors, practical servers fix the worker pool (W = 4 pthreads) and bound the accept queue (Q = 64 FIFO slots).
- **Little's Law (L = λW) & Bufferbloat:** Once all W = 4 workers are busy, extra concurrent clients (C − 4) wait in the FIFO queue rather than increasing throughput—creating application-layer bufferbloat [9]–[11].

Motivating Baseline Evidence at C = 64 (Report Table 5.3)

Median p50 = 0.53 ms vs. Tail p99 = 434.75 ms: An 820× gap between median and 99th-percentile latency in Standard mode (Retry-Aware: p50 = 0.54 ms vs. p99 = 444.17 ms) while throughput stays flat (~6.9k req/s).

Baseline p50 vs. p99 Tail Growth (Historical v4, 5 × 10 s Trials)



Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 1.1–1.2 & Table 5.3 (Historical v4 Paired Dataset)

Research Question, Engineering Objectives & Networking Scope

Twofold problem formulation, core research question, and mapping to Computer Networks principles (Report Table 1.1)

Core Research Question (Report §1.2)

- **Tail Protection Under Overload:** Can a lightweight in-server feedback controller (QLAAC) using worker utilisation u, queue occupancy q, processing latency p95, and queue-wait p95 bound p99 tail latency when C > W?
- **Quantifying the Exact Cost:** What is the measurable trade-off in successful 2xx throughput, p50/p95 latency, HTTP 503 rejections, and client Retry-After back-off?

Engineering Objectives (Report §1.3)

- **1. Raw POSIX C17 Server:** Non-blocking listener, 64-slot mutex/condvar FIFO queue, 4 pthread workers, partial recv/send/writev, and zero-copy sendfile(2).
- **2. RFC 9112 Subset & Security:** Streaming parser, decimal Content-Length (≤10 MiB), keep-alive, /health, /metrics, /api/echo, and 5-layer traversal sandbox.
- **3. Paired Evaluation & 92 Tests:** Controlled same-binary evaluation (80 v4 + 80 fresh rerun trials) and 92 automated C/Python tests.

Computer Networks Concept	Project Implementation & Significance (Report Table 1.1)
Transport Layer: TCP Lifecycle & Byte-Stream [3]	src/tcp_socket.c: listen(128), non-blocking accept4, poll deadlines, partial recv/send loops, shutdown(SHUT_WR) lingering close
Application Layer: HTTP/1.1 Framing & Status [1,2]	src/http_parser.c, http_response.c: Content-Length framing, keep-alive persistence, deterministic 400/405/411/413/414/417/431/501/505
Client–Server Concurrency & Bounded Buffering	src/thread_pool.c, connection_queue.c: 1 listener, 64-slot circular FIFO queue, 4 pre-spawned pthreads, exclusive socket ownership
Queueing Delay, Bufferbloat & Overload Control [8,9,11]	src/adaptive_controller.c, server.c: enqueue-to-dequeue wait telemetry, CAUTION fair sharing (≤16 reqs), OVERLOAD 503 + Retry-After: 5
Network Performance Measurement & Accounting	benchmark/run_experiment.py, run_reproducible_evaluation.py: strict 2xx RPS, per-trial p50/p95/p99, 503 rate, and client backoff
Network Application Security (Path Sandbox)	src/static_files.c: segment-aware '..' check, %00/backslash reject, realpath() root containment, O_NOFOLLOW + fstat inode match

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 1.2–1.5 & Table 1.1 (pp. 6–9)

Literature Context: Standards, Concurrency & Delay Control

Establishing the motivation and gap from normative IETF RFCs, server architectures, and queue-delay management (Table 2.1)

Ref	Authors & Year	Standard / Technique	Key Principle Used	Gap Addressed in Our Project
[1,2]	Fielding, Nottingham, Reschke (IETF 2022)	RFC 9112 (HTTP/1.1) & RFC 9110 (HTTP Semantics)	Normative Content-Length framing, persistent connections, 503 & Retry-After	Defines 503/Retry-After syntax but prescribes no server overload admission policy
[3]	Eddy, Ed. (IETF 2022)	RFC 9293 (Transmission Control Protocol)	Unstructured byte-stream delivery, listen backlog, half-close FIN vs RST	Requires application-level buffering, partial-I/O loops, and lingering close
[4,5]	Welsh, Culler & Brewer (SOSP 2001; USITS 2003)	SEDA Staged Event-Driven Architecture & Stage Control	Explicit bounded queues and 90th-percentile latency feedback shedding	Coupled to multi-stage event pipelines rather than a flat POSIX thread pool with keep-alive rotation
[6]	Schroeder & Harchol-Balter (ACM TOIT 2006)	SRPT Connection Scheduling Under Overload	Size-based scheduling reduces mean response time during overload	Requires known static file sizes; does not bound queue depth or shed excess arrivals
[7]	Pariag et al. (ACM EuroSys 2007)	Comparison of Thread-per-Conn, Event & Hybrid Servers	Shows concurrency tuning matters as much as server architecture	Evaluates peak capacity tuning; does not study 503 admission control or Retry-After clients
[8,9]	Nichols & Jacobson (2012); Gettys (2011)	CoDel Delay-Based AQM & Bufferbloat Analysis	Measures queueing sojourn time rather than raw queue length alone	Targets network packet queues rather than HTTP/1.1 keep-alive connection queues

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 2.1–2.3, Table 2.1 (pp. 10–11) & Section 7 References

Existing Approach (Baseline): Fixed Policy on Identical Binary

Controlled baseline: the exact same compiled C17 executable (bin/server) with adaptive_control_enabled = false

Baseline Architectural Invariants (Report §3.2)

- **Same Compiled Executable:** Launched via ./bin/server -c config/server.conf.example -p 8080 (without -a). Zero code differences in socket I/O, parser, router, or thread pool.
- **Four Fixed pthread Workers:** 4 pre-spawned POSIX worker threads (worker_threads = 4). Each worker owns an accepted connection exclusively until it closes.
- **Bounded 64-Slot FIFO Queue:** Strictly NOT an unbounded queue. src/connection_queue.c enforces capacity = 64 under mutex + condition variables (listen backlog = 128).
- **Static Keep-Alive Behaviour:** Allows up to 100 sequential requests per persistent connection with a 5-second idle timeout at all load levels.
- **Passive Queue-Full 503 Path Only:** Listener sends 503 (Retry-After: 5) only if connection_queue_push(..., 0) returns non-zero when the queue is 64/64 full. No latency or queue-wait feedback.

Baseline Request Path (Fixed Policy)

Accept Gate
Always ADMIT
(if queue < 64)

→

FIFO Queue
Capacity = 64
Wait unmonitored

→

4 Workers
Pinned ≤100 reqs
5 s idle timeout

Why queue-full 503 never triggered: In closed-loop tests with C ≤ 64 and W = 4, 4 connections are in service and at most 60 wait in the 64-slot queue (0 5xx in all 40 baseline trials).

Baseline Config & Gate (config/server.conf.example)

```
adaptive_control_enabled = false  # Baseline default
worker_threads = 4                # Fixed pthread pool
connection_queue_capacity = 64    # Bounded FIFO cap
keepalive_max_requests = 100      # Harness: 50/conn
keepalive_timeout_sec = 5         # 5s idle timeout
/* Disabled: evaluate() -> QLAAC_ACTION_ADMIT */
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 3.2 (p. 13); src/server.c; src/connection_queue.c

Baseline Bottleneck, Queue-Wait Model & Research Gap

Why static keep-alive limits cause head-of-line worker pinning and why passive queue-full refusal cannot fix it

Mechanism of Head-of-Line Pinning & Gap (Report §2.3, §5.4)

- **50-Request Keep-Alive Pinning:** Each benchmark client sends up to 50 sequential GET /health requests per connection. A worker is pinned to its connection for all 50 requests before dequeuing the next socket.
- **Service Rounds of 200 Requests:** Four workers complete 4 × 50 = 200 requests per service round while C − 4 connections wait in the FIFO queue for ⌈(C − 4) / 4⌉ rounds.
- **Why p50/p95 Stay Tiny While p99 Explodes:** Only the 1st request on a newly accepted socket (1 in 50 = 2.0%) waits in the connection queue; the next 49 requests (98%) execute immediately (~0.5 ms). Thus p50 and p95 stay ≤1.34 ms while p99 captures the 2% of requests delayed across rounds.
- **The Baseline Gap:** Passive queue-full shedding never fires at C ≤ 64 (max 60/64 queued), and static keep-alive (100 reqs) never rotates workers faster when the queue builds.

Mode	Clients C	2xx RPS (X)	Rounds ⌈(C−4)/4⌉	Predicted Wait (ms)	Observed p99 (ms)
Standard	8	6,869.56	1	29.1 ms	28.54 ± 0.72
Standard	32	6,938.70	7	201.8 ms	201.93 ± 6.43
Standard	64	6,922.30	15	433.4 ms	434.75 ± 9.41
Retry-Aware	8	6,884.42	1	29.1 ms	28.40 ± 0.22
Retry-Aware	32	6,767.70	7	206.9 ms	206.97 ± 7.90
Retry-Aware	64	6,805.68	15	440.8 ms	444.17 ± 7.13

Approximate Consistency Check (Report Table 5.5)

Predicted Wait ≈ ⌈(C − 4) / 4⌉ × (200 / X) × 1000 ms: Approximate consistency check using measured 2xx RPS (X): C = 8 differs by ~2.0% (Standard: 29.1 vs 28.54 ms) and ~2.3% (Retry-Aware: 29.1 vs 28.40 ms); C = 32 and C = 64 lie within ~1% of observed p99.

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 2.3 (p. 11), Section 5.4 & Table 5.5 (p. 33)

SECTION 4 • PROPOSED NOVELTY

Proposed Novelty: Queue-and-Latency-Aware Admission Control

Four implemented mechanisms differentiating QLAAC from the baseline within the same C17 binary (Report §3.4)

1. Dual Latency & Queue-Wait Telemetry (1,000 ms Sampler)

Four In-Process Signals: Sampler thread wakes every 1,000 ms and evaluates worker utilisation u (active/4), queue occupancy q (depth/64), 5 s rolling p95 request processing latency, and 5 s rolling p95 enqueue-to-dequeue queue wait (1,024-sample reservoirs).

2. Keep-Alive Rotation / Fair Sharing in CAUTION (≤16 Reqs)

Faster Worker Turnover Without Shedding: In CAUTION, QLAAC admits all connections (0 rejections) but caps keep-alive at ≤16 requests with a 2 s idle timeout (down from 100 reqs / 5 s), rotating waiting connections across the 4 workers ~3× faster.

3. Accept-Gate HTTP 503 + Retry-After: 5 in OVERLOAD

Pre-Queue Shedding & Cooperative Back-Off: In OVERLOAD, the listener rejects newly accepted sockets immediately with HTTP 503 Service Unavailable and Retry-After: 5 before they occupy a queue slot, while tightening held connections to ≤16 requests and 1 s idle.

4. Asymmetric Escalation & 5-Window Recovery Hysteresis

Dampening State Oscillation: Escalates immediately on single-window CAUTION/OVERLOAD triggers (or 3 consecutive sustained-pressure windows), but requires 5 consecutive healthy 1 s windows per single-step de-escalation (OVERLOAD → CAUTION → NORMAL).

Honest Scope of Contribution (Report §2.3 & §3.4): QLAAC is a measured systems integration of delay-based signals [8], admission control [5], and keep-alive rotation—not a wholly new algorithm class or universal performance improvement.

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 2.3, 3.3, 3.4 & Table 3.2 (pp. 11, 13–17)

SECTION 5 · ARCHITECTURE OR ALGORITHM

End-to-End Request-Handling & Feedback Architecture

Editable data-flow and closed-loop telemetry diagram across our C17 modules (Report Figure 3.1)

SYNCHRONOUS REQUEST PATH (SOLID ARROWS) & OVERLOAD REJECTION PATH (RED)

1. TCP Clients
Persistent HTTP/1.1
Standard / Retry-Aware

→

2. Listener
src/server.c
poll + accept4()

→

3. QLAAC Gate
adaptive_controller
evaluate() gate

→

4. FIFO Queue
connection_queue
64 slots (enq_ns)

→

5. 4 Workers
src/thread_pool.c
pthreads (deq_ns)

→

6. HTTP Parser
src/http_parser.c
Streaming RFC 9112

→

7. Router
src/router.c
/health /metrics /echo

→

8. Response
http_response.c
sendfile / writev

Early Rejection Path: If QLAAC Gate (3) is in OVERLOAD—or if FIFO Queue (4) is 64/64 full—the listener immediately writes HTTP/1.1 503 Service Unavailable with Connection: close and Retry-After: 5, then performs a 50 ms lingering close.

ASYNCHRONOUS TELEMETRY & CLOSED-LOOP FEEDBACK PATH (1,000 ms SAMPLING WINDOW)

9. Metrics Collector (src/metrics.c)
- Records 2xx–5xx counters & active workers
- 1,024-sample (5 s) proc-latency reservoir
- 1,024-sample (5 s) queue-wait reservoir

→

10. Sampler Thread (src/server.c)
- Wakes every 1,000 ms (50 ms slices)
- Reads u = active/4, q = depth/64
- Computes p95_proc_ms & p95_qwait_ms

→

11. QLAAC State & Actuation
- Updates NORMAL / CAUTION / OVERLOAD
- Steers Gate (3) & Workers (5) via get_state()
- Exports live JSON snapshot on GET /metrics

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 3.1 & Figure 3.1 (p. 12); src/server.c, src/thread_pool.c

SECTION 5 · ARCHITECTURE OR ALGORITHM

QLAAC State Machine, Thresholds & Moderate-Queue Logic

Three-state transition rules (Figure 3.2), configured thresholds (Table 3.1), and why C = 32 stays in CAUTION

NORMAL State (Green)
- Admit all sockets (if queue < 64)
- Keep-Alive: ≤ 100 reqs, 5 s idle
- Active at C = 2 & C = 8 (q < 0.20)

Escalate →

← 5 Windows

CAUTION State (Amber)
- Admit all sockets (0 HTTP 503s!)
- Fair Sharing: ≤ 16 reqs, 2 s idle
- Active at C = 32 (q = 0.44, qw ~200 ms)

Escalate →

← 5 Windows

OVERLOAD State (Red)
- Accept Gate: 503 + Retry-After: 5
- Held sockets: ≤ 16 reqs, 1 s idle
- Active at C = 64 (q = 60/64 = 0.94)

Signal / Parameter	Configured Threshold (Table 3.1)	Role in adaptive_controller_update_queue_aware() & Design Rationale
Worker utilisation u	caution ≥ 0.85; saturated ≥ 1.00	Active workers ÷ 4. Never triggers CAUTION/OVERLOAD alone; requires queue (q ≥ 0.50/0.70) or latency evidence
Queue occupancy q	growing ≥ 0.70 (45/64); critical ≥ 0.90 (58/64)	Current FIFO depth ÷ 64. q ≥ 0.90 triggers immediate OVERLOAD; q ≥ 0.70 gates queue_wait_overload
Processing latency p95	elevated ≥ 50 ms; critical ≥ 150 ms	95th percentile of worker request service time over last 5.0 s (1,024-sample rolling reservoir)
Queue-wait p95 (gated on q ≥ 0.20)	elevated ≥ 25 ms; critical ≥ 100 ms	95th percentile of enqueue-to-dequeue wait over last 5.0 s; gated on has_queue_presence (q ≥ 0.20) to ignore stale samples
Why C = 32 Enters CAUTION, Not OVERLOAD	q = 28/64 = 0.44 (< 0.70), qw_p95 ~200 ms	**Because q < 0.70, critical queue wait classifies as head-of-line keep-alive delay → CAUTION (≤16 reqs, 0 rejections)**
Sustained-pressure & Recovery windows	3 pressure windows; 5 recovery windows	3 consecutive pressure ticks escalate to OVERLOAD; 5 consecutive healthy 1 s ticks required per single-step de-escalation

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 3.3, Table 3.1 & Figure 3.2 (pp. 14–16); src/adaptive_controller.c

QLAAC Algorithm & Genuine C17 Source Implementation

Exact correspondence between the controller predicates, listener accept gate, and worker loop (Appendix A.1–A.3)

1. Controller Update: src/adaptive_controller.c (App. A.1)

```
bool has_q = (queue_occupancy_ratio >= 0.20);
bool qw_elev = (p95_qw >= cfg->queue_wait_caution_ms && has_q);
bool qw_crit = (p95_qw >= cfg->queue_wait_overload_ms && has_q);
bool sust_cand = (worker_saturated && q_ratio >= 0.50) ||
    (queue_growing && (qw_elev || latency_elevated));
bool qw_overload = qw_crit &&
    (queue_growing || state == QLAAC_STATE_OVERLOAD);
bool is_overload = queue_critical || latency_critical ||
    qw_overload || (worker_saturated && queue_growing) ||
    sustained_pressure_overload;
bool w_stress_q = (worker_caution || worker_saturated) &&
    (q_ratio >= 0.50 || qw_elev || latency_elevated);
bool is_caution = is_overload || queue_growing ||
    qw_elev || qw_crit || latency_elevated || w_stress_q;
```

2. Listener Accept Gate: src/server.c (Appendix A.2)

```
QlaacDecision d = adaptive_controller_evaluate(&ctx->ctrl);
if (d == QLAAC_ACTION_REJECT_OVERLOAD) {
    metrics_record_connection_rejected(&ctx->metrics);
    const char *msg_503 = "HTTP/1.1 503 ... Retry-After: 5";
    if (tcp_send_all(client.fd, msg_503, len, dl, &s) == 0)
        metrics_record_overload_response(&ctx->metrics, 503, s);
    tcp_lingering_close(&client.fd, 50); continue;
}
```

3. Worker Fair Sharing: src/thread_pool.c (Appendix A.3)

```
/* Side-effect-free query: does NOT increment shed count */
QlaacState qs = adaptive_controller_get_state(&ctx->ctrl);
if (adaptive_on && qs == QLAAC_STATE_OVERLOAD) {
    keepalive_timeout_sec = 1; max_requests = min(cfg_max, 16);
} else if (adaptive_on && qs == QLAAC_STATE_CAUTION) {
    keepalive_timeout_sec = 2; max_requests = min(cfg_max, 16);
} else { /* NORMAL: 5 s idle, up to 100 requests */ }
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 3.3 (pp. 14–16) & Appendix A.1–A.3 (pp. 41–42)

Raw POSIX TCP Socket I/O, Byte-Stream Framing & sendfile(2)

Non-blocking descriptors, poll-sliced deadlines, pipelined byte preservation, and zero-copy sendfile fallback

Transport & Buffering in src/tcp_socket.c & http_parser.c

- **Non-Blocking Sockets & Backlog:** listen(fd, 128) with SO_REUSEADDR, IPV6_V6ONLY, non-blocking accept4(..., SOCK_NONBLOCK | SOCK_CLOEXEC), and TCP_NODELAY.
- **TCP Byte-Stream Buffering:** TCP (RFC 9293) has no message boundaries: one recv() can return a partial header, partial POST body, or pipelined requests. ConnectionBuffer shifts unconsumed pipelined bytes to index 0 via memmove().
- **Poll-Sliced Partial Read/Write Loops:** wait_poll() slices waits into 50 ms increments checking atomic stop_requested; tcp_recv_some(), tcp_send_all(MSG_NOSIGNAL), and tcp_sendv_all(writev) loop on EINTR and EAGAIN until completion.
- **Zero-Copy Linux sendfile(2) & Fallback:** tcp_sendfile_all() streams static files in ≤1 MiB kernel chunks. If sendfile fails mid-transfer, lseek(file_fd, offset, SEEK_SET) synchronizes the offset before falling back to a 16 KiB read/send loop.
- **Orderly Lingering Close:** tcp_lingering_close() calls shutdown(fd, SHUT_WR) and drains residual client bytes for up to 200 ms (50 ms on 503) to prevent TCP RST truncation.

Excerpts: src/tcp_socket.c & src/http_parser.c (conn_buffer_consume)

```
void conn_buffer_consume(ConnectionBuffer *cb, size_t n) {
    if (n >= cb->length) cb->length = 0;
    else { memmove(cb->buf, cb->buf + n, cb->length - n);
           cb->length -= n; }
}
int tcp_send_all(int fd, const void *buf, size_t len, uint64_t dl) {
    const char *p = (const char *)buf; size_t rem = len;
    while (rem > 0) {
        ssize_t n = send(fd, p, rem, MSG_NOSIGNAL);
        if (n > 0) { p += n; rem -= (size_t)n; continue; }
        if (errno == EAGAIN && wait_poll(fd, POLLOUT, dl, 0) < 0)
            return TCP_ERR_IO;
    }
    return 0;
}
/* Zero-copy sendfile(2) with offset-preserving fallback */
off_t offset = lseek(file_fd, 0, SEEK_CUR);
ssize_t n = sendfile(sock_fd, file_fd, &offset, chunk);
if (n < 0) { lseek(file_fd, offset, SEEK_SET); /* read/send */ }
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 1.1, 4.3 & Table 4.4 (pp. 6, 20–25); src/tcp_socket.c, src/http_parser.c

Bounded FIFO Queue & Fixed pthread Worker Pool

Producer-consumer circular buffer, monotonic queue-wait timestamps, exclusive socket ownership, and fast shutdown

Synchronisation & Ownership (src/connection_queue.c, thread_pool.c)

- **Circular FIFO Buffer (Cap 64):** ConnectionQueue holds head, tail, count, capacity (64), and is_shutdown, guarded by pthread_mutex_t lock and condvars not_empty & not_full.
- **Non-Blocking Listener Push:** Listener calls connection_queue_push(&ctx->queue, &client, 0). If count == 64, timeout_ms == 0 returns -1 immediately so the accept loop never blocks and replies 503.
- **Monotonic Queue-Wait Timing:** Push stamps enqueued_ns = monotonic_ns(); pop stamps dequeued_ns = monotonic_ns(). Worker records (dequeued_ns - enqueued_ns) / 1e6 ms via metrics_record_queue_wait().
- **Exclusive Socket Ownership:** Once popped, a worker thread owns conn.fd exclusively for its entire keep-alive lifetime—zero mutex contention during parsing or sending.
- **Prompt Graceful Shutdown:** Workers pop with worker_idle_timeout_sec (2 s) and poll recv in 50 ms slices checking atomic stop_requested, terminating in < 1.5 s on SIGTERM.

Genuine Excerpts: src/connection_queue.c & src/thread_pool.c

```
int connection_queue_push(ConnectionQueue *q,
        const ClientConnection *c, int timeout_ms) {
    pthread_mutex_lock(&q->lock);
    while (q->count == q->capacity) {
        if (q->is_shutdown) { unlock(&q->lock); return -2; }
        if (timeout_ms == 0) { unlock(&q->lock); return -1; }
    }
    q->buffer[q->tail] = *c;
    if (q->buffer[q->tail].enqueued_ns == 0)
        q->buffer[q->tail].enqueued_ns = monotonic_ns();
    q->tail = (q->tail + 1) % q->capacity; q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock); return 0;
}
/* In worker_thread_routine (src/thread_pool.c): */
int rc = connection_queue_pop(pool->queue, &conn, t_ms);
uint64_t qw_ns = (deq_ns >= enq_ns) ? (deq_ns - enq_ns) : 0;
metrics_record_queue_wait(&ctx->metrics, (double)qw_ns / NS_PER_MS);
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.3 (pp. 20–21); src/connection_queue.c, src/thread_pool.c

Streaming HTTP/1.1 Parser, Framing & Protocol Scope

Strict RFC 9112/9110 framing, anti-smuggling checks, and documented HTTP/1.1 subset in src/http_parser.c

Condition / Violation Checked in http_parse_request()	ParserStatus Enum	HTTP Status
Incomplete header (no \r\n\r\n) or body < Content-Length	PARSER_NEED_MORE_DATA	(Keep reading)
Missing/duplicate Host, TE+CL conflict, repeated CL, bare CR/LF, space before ':'	PARSER_ERR_BAD_REQUEST	400 Bad Request
POST without Content-Length header	PARSER_ERR_LENGTH_REQUIRED	411 Length Req
Content-Length > max_body_size (10 MiB) or ERANGE overflow	PARSER_ERR_BODY_TOO_LARGE	413 Content Too Large
Request-URI length ≥ 2,048 bytes (max accepted = 2,047 B)	PARSER_ERR_URI_TOO_LONG	414 URI Too Long
Expect: 100-continue header present (deliberately unsupported)	PARSER_ERR_EXPECTATION_FAILED	417 Expectation Failed
Header name ≥128 B, value ≥2,048 B, >64 fields, or block > max_header_size	PARSER_ERR_HEADERS_TOO_LARGE	431 Hdr Too Large
Transfer-Encoding: chunked or unknown method token (e.g. TRACE)	PARSER_ERR_NOT_IMPLEMENTED	501 Not Impl
Unsupported HTTP/* version (neither HTTP/1.1 nor HTTP/1.0)	PARSER_ERR_VERSION_NOT_SUPPORTED	505 Version Not Sup

Excerpt: src/http_parser.c (Framing & Anti-Smuggling)

```
/* 1. Strict 1*DIGIT Content-Length (RFC 9110 §8.6) */
if (cl_found) { free(hdr_buf); return PARSER_ERR_BAD_REQUEST; }
cl_found = true; req->has_content_length = true;
for (const char *p = hval; *p != 0; p++) {
    if (!isdigit((unsigned char)*p)) {
        free(hdr_buf); return PARSER_ERR_BAD_REQUEST; }
}
unsigned long long val = strtoull(hval, &endptr, 10);
if (errno == ERANGE || val > max_body_size)
    return PARSER_ERR_BODY_TOO_LARGE;
/* 2. Framing invariants (RFC 9112 §3.2, §6.3) */
if (req->is_http_11 && !host_found)
    return PARSER_ERR_BAD_REQUEST;
if (req->has_transfer_encoding && req->has_content_length)
    return PARSER_ERR_BAD_REQUEST; /* TE+CL defense */
if (req->has_transfer_encoding)
    return PARSER_ERR_NOT_IMPLEMENTED;
if (req->has_expect_100) return PARSER_ERR_EXPECTATION_FAILED;
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 1.4, 4.3 & Table 4.4 (pp. 8, 22, 24–25); src/http_parser.c

Endpoint Routing, Static-File Delivery & Sandbox Security

Verified route table (src/router.c), query-string separation, and 5-layer path-traversal defense (src/static_files.c)

Route / Target	Allowed Methods	Disallowed Method Status	Implemented Handler Behaviour (src/router.c)
/health or /status	GET, HEAD	405 Method Not Allowed	200 OK JSON: {"status":"healthy", "server":"HTTP-Server-C17/1.0.0", "protocol":"HTTP/1.1"}
/metrics	GET, HEAD	405 (or 404 if disabled)	200 OK JSON snapshot of counters, queue, workers, RPS, proc/q_wait percentiles, QLAAC state (lat=-1.0 excluded)
/api/echo	POST only	405 Method Not Allowed	200 OK echoing req->body (up to 10 MiB) with incoming Content-Type (default text/plain; charset=utf-8)
/ or /<static-file>	GET, HEAD	405 Method Not Allowed	Resolves inside public_root (./public), serves default index.html, streams via zero-copy sendfile(2)

5-Layer Path-Traversal & Sandbox Defense (src/static_files.c)

- **1. Segment-Aware Dot-Dot Check:** has_traversal_segment() blocks '..' and '%2e%2e' path segments (403) while permitting benign filenames like release..txt.
- **2. Strict URL Percent-Decoding:** url_decode() rejects incomplete hex escapes, NUL bytes (%00), and backslashes ('\\') with 400 Bad Request.
- **3. Post-Decode Traversal Re-Check:** Re-runs has_traversal_segment(decoded_uri) to block encoded dot-dot bypasses (403).
- **4. Canonical Root Sandbox (realpath):** Resolves realpath(public_root) and realpath(candidate), verifying canonical_target starts with canonical_root + '/'.
- **5. Symlink & TOCTOU Protection:** Opens with O_RDONLY | O_CLOEXEC | O_NOFOLLOW and verifies fstat() st_dev/st_ino and S_ISREG match stat().

Excerpt: src/static_files.c (Sandbox & O_NOFOLLOW Check)

```
/* 1. Canonical target must stay inside canonical_root */
size_t rlen = strlen(canonical_root);
if (strncmp(canon_target, canonical_root, rlen) != 0 ||
    (canon_target[rlen] != '/' && canon_target[rlen] != 0))
    return build_403_forbidden(res);
/* 2. Open with O_NOFOLLOW & verify fstat dev/inode */
int fd = open(final_file, O_RDONLY | O_CLOEXEC | O_NOFOLLOW);
if (fstat(fd, &open_st) != 0 || !S_ISREG(open_st.st_mode) ||
    open_st.st_dev != st.st_dev || open_st.st_ino != st.st_ino)
{ close(fd); return build_403_forbidden(res); }
```

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.3 & Table 4.4 (pp. 20, 22, 24); src/router.c, src/static_files.c

Build Setup, Codebase Map & Testbed Environments

12 C17 modules (3,569 snapshot / 3,573 ZIP lines), config parameters, and strict separation of test environments

Build Stack, Config & Live-Demo Setup

- **Compiler & Flags:** GCC C17 (-std=c17 -D_POSIX_C_SOURCE=200809L -O2 -pthread) with -Wall -Wextra -Wpedantic -Wformat=2 -Wshadow; gcc -fanalyzer 0 warnings.
- **Config & Buffer Limits:** config/server.conf.example sets worker_threads=4, queue=64, keepalive=100/5s, and max_header_size=io_buffer_size=8,192 B (compiled fallback default in src/server.c is 16,384 B).
- **Live-Demo Setup:** Two-terminal workflow on free port (e.g. -p 18080): Terminal 1 runs foreground ./bin/server (stopped via Ctrl+C before adding -a or -t 2); Terminal 2 runs curl & functional burst (full commands in Demo Runbook).

Codebase Layer (Table 4.3)	Lines (Report / ZIP)	Key Files & Roles
src/*.c (12 C17 files)	3,569 / 3,573*	main (140), server (420), tcp_socket (486), queue (194), pool (285), parser (479/483*), resp (332), router (205), static (287), qlaac (299), metrics+util (442)
include/*.h (11 headers)	1,144	Public interfaces, structs, constants (URI/header limits)
public/index.html	1,470	Single-page HTML5/CSS/JS live telemetry dashboard
tests/ (5 C, 5 Python)	1,478 C + 889/1,251* Py	44 C unit tests + 48 Python tests = 92 passing tests
benchmark/*.py (6 scripts)	1,808 / 1,875*	run_experiment.py, run_reproducible_evaluation.py (561/628*)

Environment ID (Table 4.1 & App. B.7)	Platform / CPU / OS / Toolchain	Strict Role & Separation Rule
1. Historical v4 Benchmark Host (Chapter 5 Primary)	Windows 11 host running Linux under WSL2, loopback 127.0.0.1:8080 (CPU/thermal metadata not in CSVs)	**Produced the 80 paired v4 trials in Chapter 5 (Tables 5.3–5.5, B.1–B.3)**
2. Earlier Workstation Audit (Prior Report Revision)	Intel Core Ultra 7 155H (22 vCPUs), 7.5 GiB RAM, Linux 6.18.33.2 WSL2, GCC 15.2.0, Python 3.14.4	Reported in prior revision; not linked to v4 CSV headers
3. Report-Revision Verification Sandbox (Oct 2026)	Ubuntu 24.04 container, Linux 6.18.44, 1 logical CPU, GCC 13.3.0, Python 3.12.3	Used ONLY for 80/92 automated tests, ASan/UBSan/TSan, and CSV stat audit — NO benchmarks
4. Fresh Verification-Host Rerun (Appendix B.7, 21 Oct)	Virtualized AMD EPYC 9V74, Linux 6.18.55, 5 visible CPUs (cgroup 4.0 CPU-eq), GCC 14.2.0, Port 18080	**Produced the separate 80-trial fresh rerun in Appendix B.7 (Tables B.7–B.8); never pooled with v4**

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.1 Table 4.1 (p. 18), Table 4.3 (pp. 20–21), App. B.2 & B.7

Key Implementation Challenges & Verified Solutions

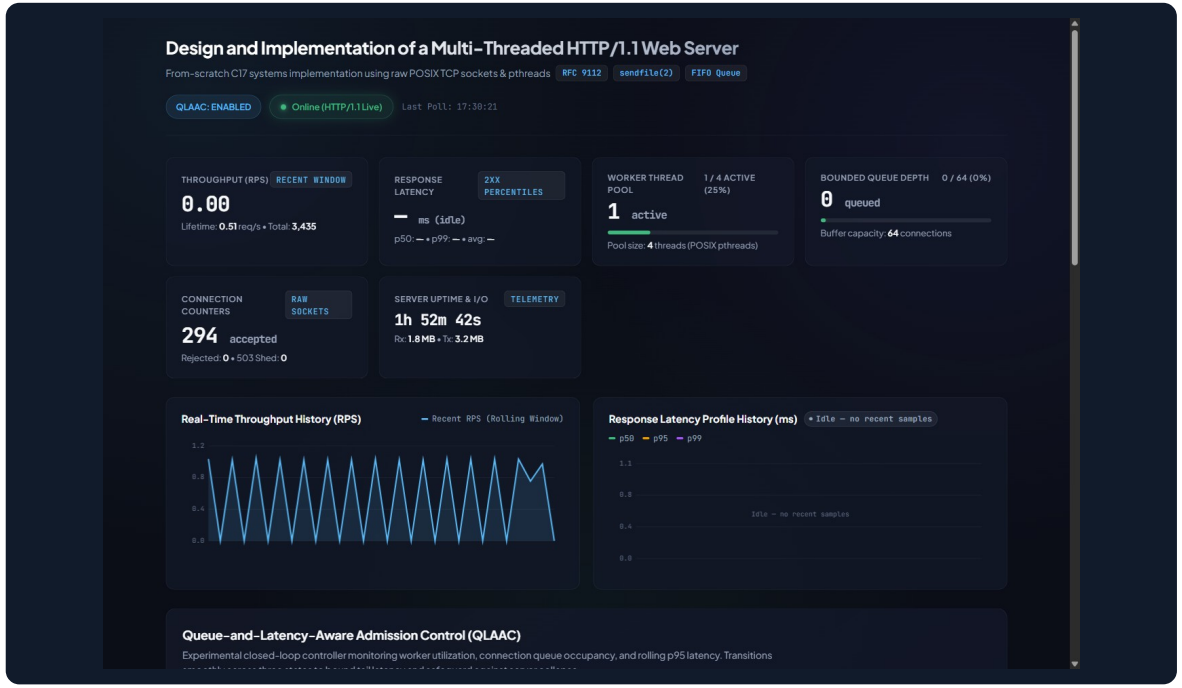
How systematic auditing and unit/integration testing resolved protocol, controller, and harness defects (Table 4.4)

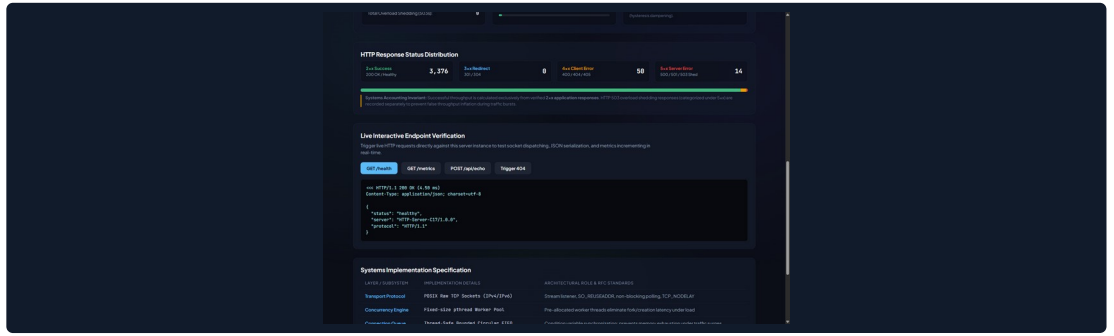
Implementation Challenge Encountered	Root Cause Identified	Implemented Solution & Regression Test Evidence
1. Early benchmark scripts inflated QLAAC throughput	Counted any 'HTTP/1.1' response as success, including fast 503 rejections	Rewrote run_experiment.py to parse status codes, count 2xx/3xx/4xx/5xx/timeouts separately; throughput = 2xx only (test_benchmark_accounting.py)
2. QLAAC collapsed throughput at C=8 (2,129 RPS) & C=32 (1,127 RPS)	v1 CAUTION set max_requests=1 & triggered on worker saturation alone; v3 triggered OVERLOAD on queue_wait ≥100 ms at q=0.44	Capped CAUTION keep-alive at 16 reqs (2 s); required queue/latency evidence with worker saturation; gated queue_wait_overload on q ≥ 0.70 → 0 rejections at C=32 in all 10 v4 trials
3. Sequential benchmark runs drifted (C=2: 21,653 vs 13,149 RPS)	Running all 40 standard trials then 40 retry-aware trials 502 s later suffered host thermal/clock drift & state carryover	Built paired interleaved runner (run_reproducible_evaluation.py): alternates Baseline/QLAAC order per pair on fresh server processes
4. QLAAC shed counter overcounted vs 503s sent	Workers called evaluate() inside keep-alive loop, incrementing total_shed_requests without sending 503	Added side-effect-free adaptive_controller_get_state() for workers; separated qlaac_shed_requests, total_connections_rejected, and total_overload_503_sent
5. Queue wait invisible to controller; 0-2xx trials reported 0.00 ms	Latency timer started only after worker dequeue; empty 2xx list defaulted to 0.0	Added enqueued_ns/dequeued_ns & p95_queue_wait_ms reservoir; report N/A when success_2xx == 0 (test_zero_success_reports_na_latency)
6. Server-dependent Python tests skipped silently or hit wrong port	Running unittest without server skipped 21/22 tests; TCP-only probe matched unrelated port-8080 service	Created scripts/run_all_tests.sh; refined wait_for_server() to verify /health JSON identity + 12 readiness tests (test_readiness_check.py)

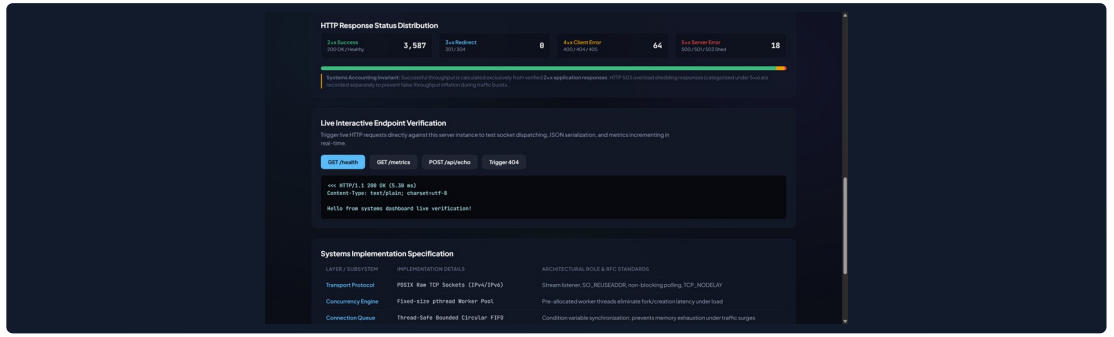
Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.4, Table 4.4 (pp. 23–25) & Appendix B.4–B.8

Genuine Runtime Captures: Dashboard, /health & /api/echo

Unaltered screenshots 01, 02, and 03 illustrating live telemetry and endpoint verification (Report Fig. 4.1, B.1, B.2)





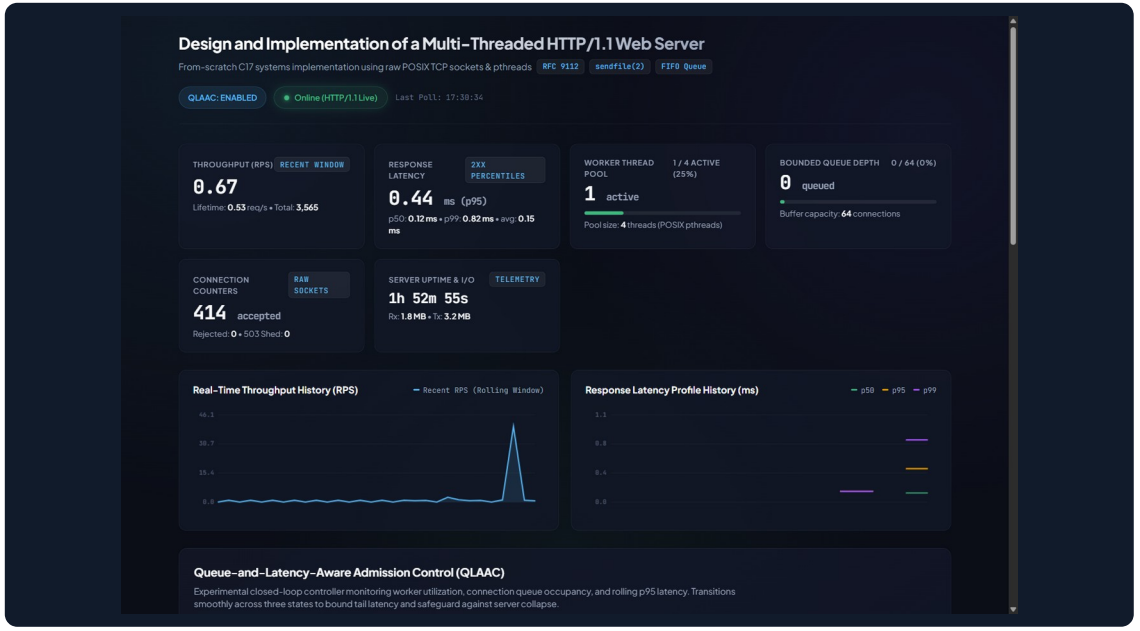


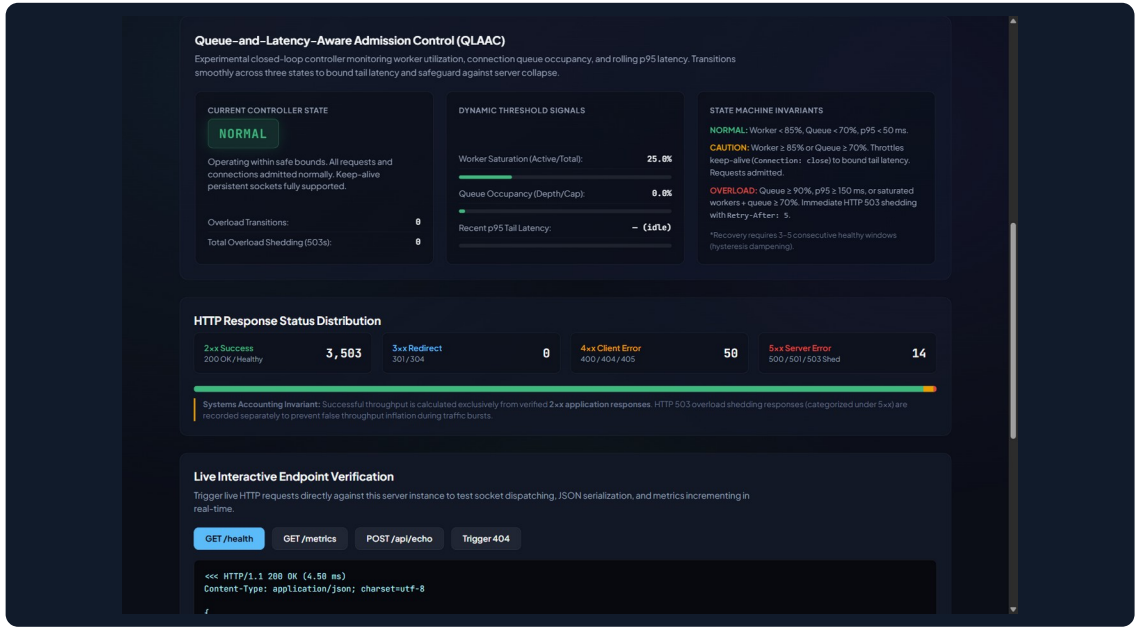
- **01_dashboard_overview.png (Left):** Shows QLAAC: ENABLED, 1/4 active workers (25%), 0/64 queue, 294 accepted, 3,435 completed. Note: '— ms (idle)' means no 2xx samples in the last 5 s window (never 0.00 ms latency); captured before the queue-wait p95 widget was added to public/index.html.
- **02_health_check.png & 03_post_echo.png (Right):** Interactive console verifying GET /health → 200 OK (4.50 ms, JSON identity 'HTTP-Server-C17/1.0.0') and POST /api/echo → 200 OK (5.30 ms, echoing Content-Length payload). Functional checks, not benchmark runs.

Source: CN_Project_Final_Report_Final_Submission.pdf, Fig. 4.1 (p. 23) & Appendix B.1 Figs. B.1–B.2 (pp. 44–45); results/screenshots/01–03.png

Genuine Runtime Captures: Burst Telemetry & QLAAC Panel

Unaltered screenshots 04 and 05 with explicit scope qualifications (Report Appendix B.1 Figs. B.3 & B.4)





04_live_metrics.png — Functional Burst (Not a Benchmark)

Populated 5 s Latency Reservoir: Shows 0.67 RPS, 414 accepted connections, and populated 2xx percentiles (p50: 0.12 ms, p95: 0.44 ms, p99: 0.82 ms, avg: 0.15 ms) after a short manual burst—used only to verify runtime telemetry, not for Chapter 5 claims.

05_qlaac_load_test.png — Idle NORMAL State (Not OVERLOAD)

Honest Visual Qualification: Despite the filename, this capture shows CURRENT CONTROLLER STATE: NORMAL at idle (Worker Saturation: 25.0%, Queue: 0.0%, p95: — idle, 0 transitions, 0 shed)—never claimed as evidence of CAUTION or OVERLOAD.

Source: CN_Project_Final_Report_Final_Submission.pdf, Appendix B.1 Figs. B.3–B.4 (pp. 45–46); results/screenshots/04–05.png

Experimental Scenarios & Paired Trial Methodology

Concurrency grid (C = 2, 8, 32, 64), client retry modes, paired fresh-process design, and separate datasets

Methodology Dimension	Configured Design in benchmark/run_reproducible_evaluation.py (Tables 4.2 & 5.2)
Target Workload & Transport	GET /health (returns 89-byte JSON payload) over local loopback (127.0.0.1); up to 50 sequential requests per persistent connection
Four Concurrency Scenarios (C)	C = 2 (S1 Low: 0 queued < 4 workers), C = 8 (S2a Medium: 4 queued = 1 round), C = 32 (S2b Med-High: 28 queued = 44% depth, 7 rounds), C = 64 (S3 High: 60 queued = 94% depth, 15 rounds)
Two Client Retry Modes	Standard client: reconnects immediately after HTTP 503. Retry-Aware client: parses Retry-After header (sleeps 5 s; 1 s fallback) and logs total_backoff_sec
Paired Trial Protocol & Duration	5 matched pairs (t = 1..5) × 10.0 s timed duration per C and client mode = 40 trials per mode, 80 timed trials per dataset
Alternating Execution Order	Odd pairs (t = 1, 3, 5) run Baseline first then QLAAC; Even pairs (t = 2, 4) run QLAAC first then Baseline to cancel thermal/clock drift
Per-Trial Process Isolation	Fresh bin/server process spawned per trial → 1.0 s warm-up at C = 2 → 0.2 s pause → 10.0 s timed trial → 0.25 s /metrics timeline polling
Two Strictly Separate Datasets	1. Historical Paired v4 (historical_v4_20261019, WSL2 host, Port 8080 — Chapter 5 primary). 2. Fresh Verification Rerun (fresh_rerun_20261021, AMD EPYC 9V74, Port 18080 — Appendix B.7). Never pooled!

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.2 Table 4.2 (pp. 18–19), Section 5.2 Table 5.2 (p. 27) & Appendix B.7

Performance Metrics, Status Accounting & Joint Interpretation

Exact metric formulas (Report Table 5.1) and why throughput, latency, 503s, and back-off must be interpreted together

Metric (Unit)	Exact Harness Definition (benchmark/run_experiment.py)	Improvement Convention & Interpretation Rule
Successful Throughput (req/s)	success_2xx ÷ duration_sec (10.0 s); strictly excludes HTTP 503s, 4xx, and timeouts	Improvement (%) = (QLAAC − Base) / Base × 100 [Positive (+) = Higher 2xx RPS = Better]
2xx Latency p50, p95, p99 (ms)	Per-trial index int(p × (n − 1)) of sorted 2xx response times (sendall to last byte); mean ± sample SD (n − 1) over 5 trials	Table 5.3 Improvement (%) = (Base − QLAAC) / Base × 100 [Positive (+) = Lower latency = Better]; Table 5.4 Signed Δ = −Imp%
Attempted vs. Completed Reqs	Total requests sent by all C threads vs. completed HTTP responses (0 timeouts & 0 transport errors in all trials)	Closed-loop offered load varies with response speed: fast 503 reconnects inflate attempts; Retry-After sleep reduces attempts
HTTP 503 Rejections & Rate (%)	http_5xx per trial (all 5xx in benchmarks are 503). Table 5.4 Rejection Rate = mean(503s) ÷ mean(attempts) × 100	Quantifies admission shedding cost (verify_benchmark_statistics.py also reports mean of 5 per-trial ratios)
Client Back-Off Time (s)	total_backoff_sec: sum of Retry-After sleep durations across all C client threads per 10 s trial (max = C × 10 = 640 s)	Retry-Aware mode only: separates client-side Retry-After sleep from server-side 2xx socket latency

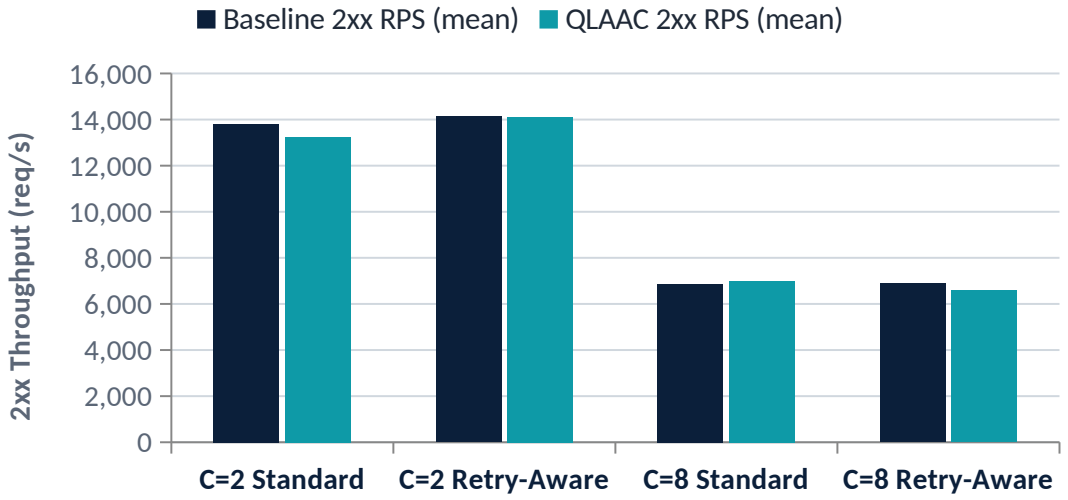
Why Throughput, Latency, Rejections & Back-Off Must Be Interpreted Together (Report §5.1, p. 26)

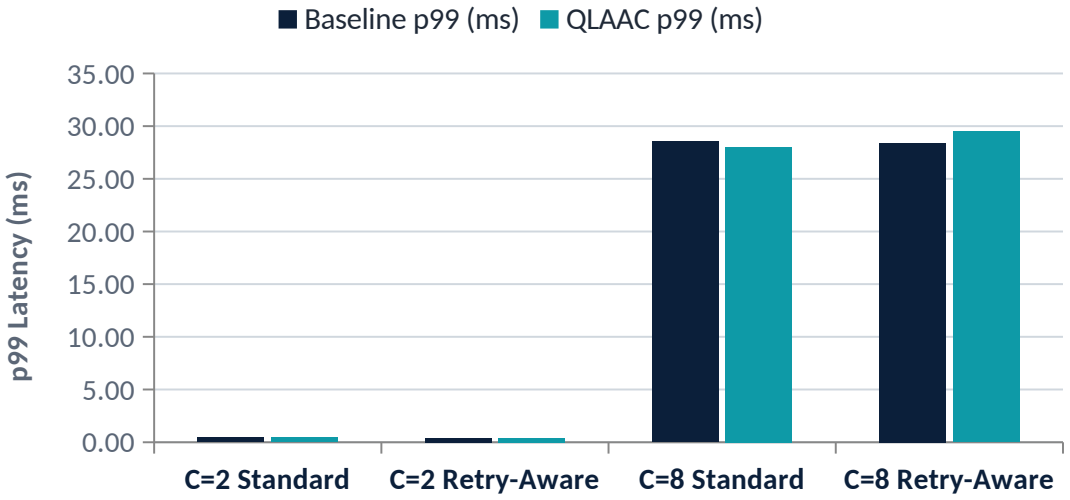
Survivorship Bias in 2xx Latency Percentiles: Latency percentiles (p50, p95, p99) are computed exclusively over admitted HTTP 2xx responses (excluding TCP connect time and Retry-After sleep). When QLAAC sheds traffic or triggers client backoff, a lower p99 describes only the surviving 2xx requests and never proves a better overall result unless read alongside 2xx throughput, 503 rejection rate, and client back-off time.

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 5.1 Table 5.1 (p. 26), Note to Table 5.3 (p. 29) & Appendix A.5

Historical v4 Results at Low & Medium Load (C = 2 and C = 8)

QLAAC remains in NORMAL state across all samples with zero 503 rejections and comparable operation (Tables 5.3 & 5.4)



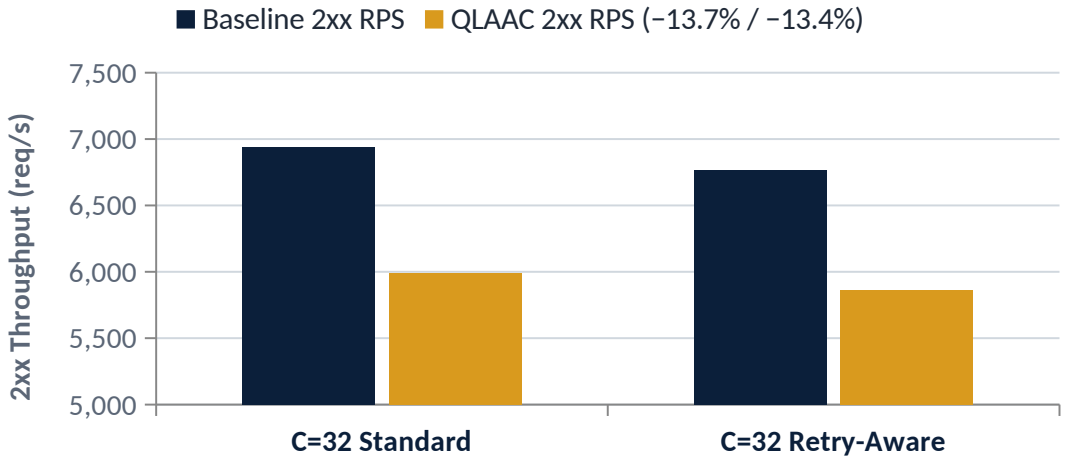


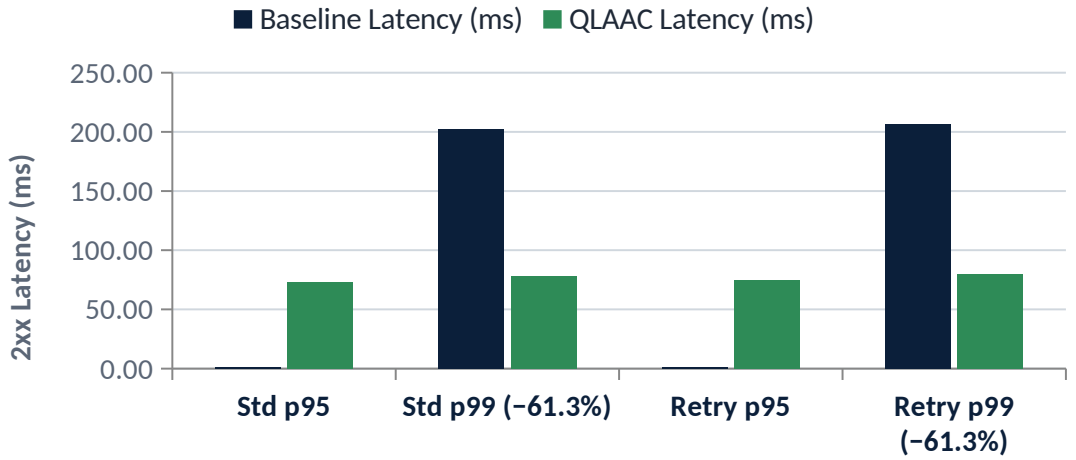
Scenario & Mode (v4 Paired, n = 5)	Baseline 2xx RPS (mean ± SD)	QLAAC 2xx RPS (mean ± SD)	Δ RPS	Baseline p50 / p95 / p99 (ms)	QLAAC p50 / p95 / p99 (ms)	503 / State
S1 Low (C=2) — Standard	13,806.78 ± 752.14	13,230.22 ± 566.31	−4.2%	0.11 / 0.26 / 0.43	0.12 / 0.28 / 0.46	0 / NORMAL
S1 Low (C=2) — Retry-Aware	14,136.06 ± 1,867.40	14,115.42 ± 856.21	−0.1%	0.11 / 0.25 / 0.41	0.11 / 0.25 / 0.42	0 / NORMAL
S2a Med (C=8) — Standard	6,869.56 ± 196.87	6,989.02 ± 270.05	+1.7%	0.53 / 1.30 / 28.54	0.52 / 1.29 / 28.02	0 / NORMAL
S2a Med (C=8) — Retry-Aware	6,884.42 ± 78.47	6,587.70 ± 219.70	−4.3%	0.53 / 1.29 / 28.40	0.55 / 1.39 / 29.49	0 / NORMAL

Source: Dataset: historical_v4_20261019 (5 × 10 s paired trials/cell, mean ± SD); Final Report Tables 5.3 & 5.4 (pp. 27–29)

Historical v4 Results at C = 32: CAUTION Fair-Sharing Trade-Off

61.3% p99 tail-latency reduction in both client modes with ZERO HTTP 503 rejections (Report Tables 5.3 & 5.4)



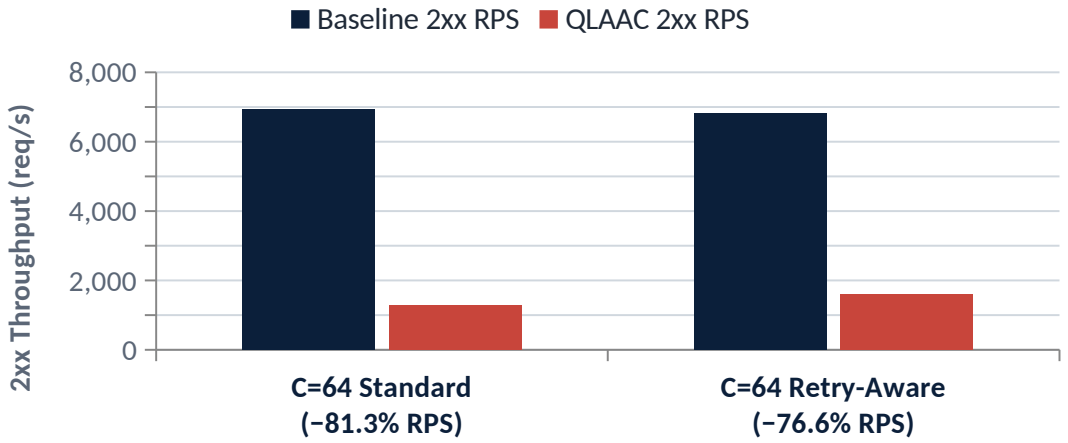


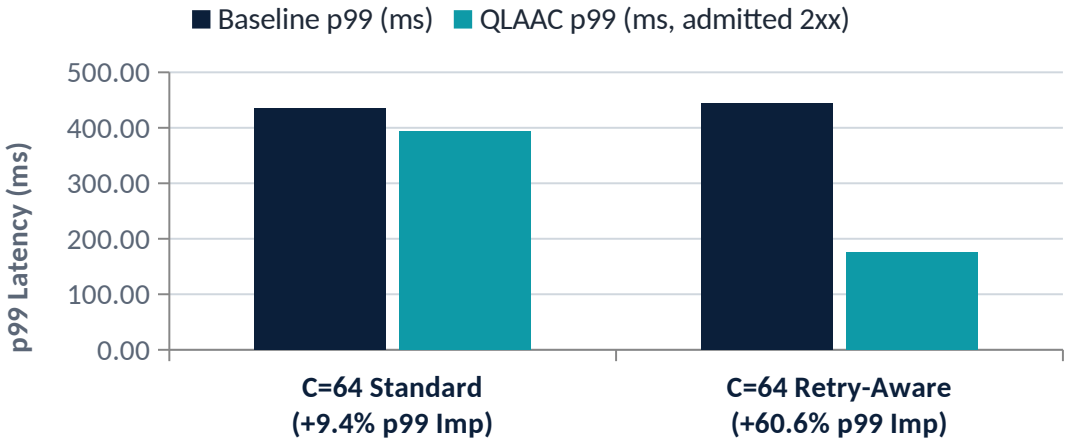
Metric (C = 32, n = 5 × 10 s)	Standard Baseline	Standard QLAAC	Std Imp% (Δ)	Retry-Aware Baseline	Retry-Aware QLAAC	Retry Imp% (Δ)
Successful 2xx Throughput (req/s)	6,938.70 ± 249.64	5,988.46 ± 101.62	**−13.7% (−13.7%)**	6,767.70 ± 270.04	5,859.92 ± 91.19	**−13.4% (−13.4%)**
p50 / Mean 2xx Latency (ms)	0.52 / 4.65 ms	0.64 / 4.86 ms	−21.1% (+21.1% p50)	0.54 / 4.76 ms	0.65 / 4.97 ms	−21.4% (+21.4% p50)
p95 2xx Latency (ms)	1.31 ± 0.06 ms	72.99 ± 1.25 ms	**−5454.8% (+5454.8%)**	1.34 ± 0.07 ms	74.65 ± 1.24 ms	**−5479.2% (+5479.2%)**
p99 Tail Latency (ms)	201.93 ± 6.43 ms	**78.17 ± 1.19 ms**	**+61.3% (−61.3%)**	206.97 ± 7.90 ms	**80.15 ± 1.84 ms**	**+61.3% (−61.3%)**
HTTP 503 Rejections & Rate (%)	0.0 ± 0.0 (0.00%)	**0.0 ± 0.0 (0.00%)**	0 shed (CAUTION)	0.0 ± 0.0 (0.00%)	**0.0 ± 0.0 (0.00%)**	0 shed / 0.00 s backoff

Source: Dataset: historical_v4_20261019 (5 × 10 s paired trials/cell, mean ± SD); Final Report Tables 5.3 & 5.4 (pp. 28–29)

Historical v4 Results at C = 64: OVERLOAD & Client Divergence

Why accept-gate 503 shedding fails under immediate-reconnect clients but cuts p99 by 60.6% under Retry-After clients



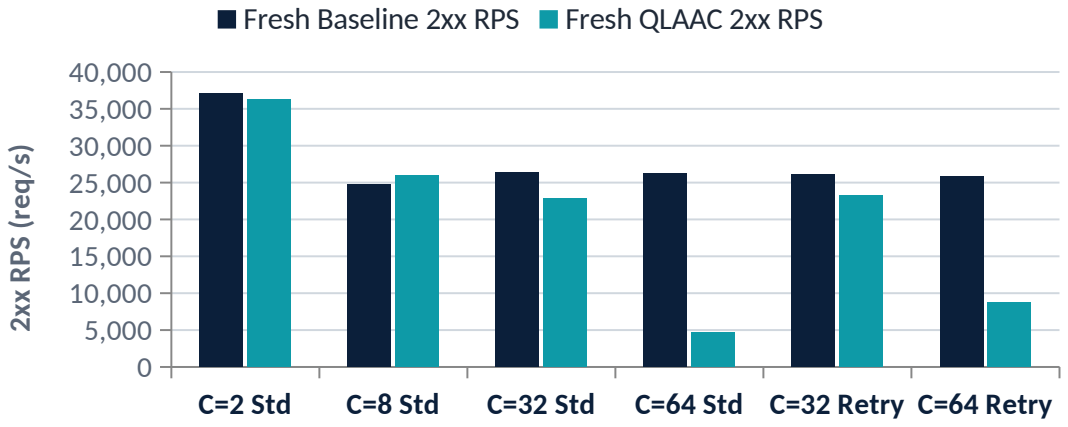


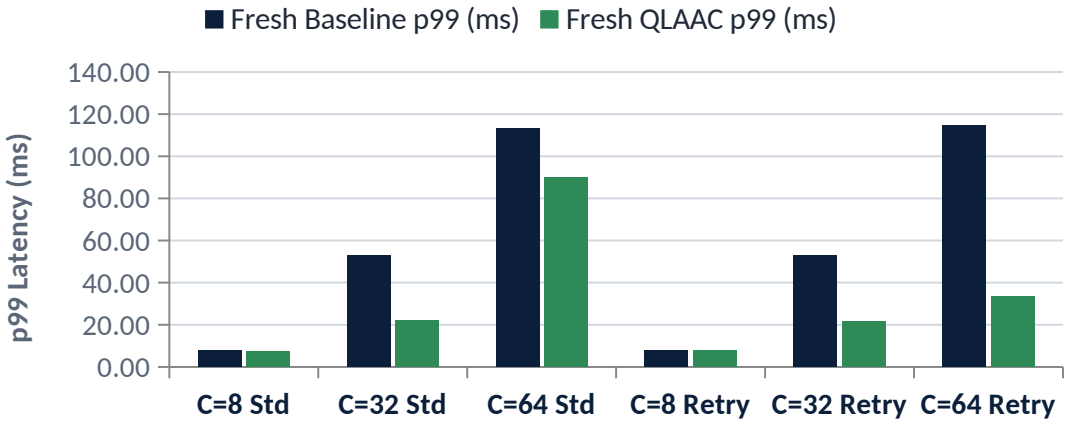
Metric (C = 64, n = 5 × 10 s)	Standard Baseline	Standard QLAAC	Std Imp% / Note	Retry-Aware Baseline	Retry-Aware QLAAC	Retry Imp% / Note
Successful 2xx Throughput (req/s)	6,922.30 ± 150.62	**1,295.26 ± 44.72**	**−81.3% (Collapse)**	6,805.68 ± 116.05	**1,591.48 ± 528.46**	**−76.6% (Backoff cost)**
p50 / p95 2xx Latency (ms)	0.53 / 1.33 ms	0.65 / 13.74 ms	p95 −936.2% (+10.4×)	0.54 / 1.33 ms	0.59 / 29.11 ± 45.62	p95 −2095% (high SD)
p99 Tail Latency (admitted 2xx)	434.75 ± 9.41 ms	393.73 ± 21.75 ms	**+9.4% (−9.4% Δ)**	444.17 ± 7.13 ms	**175.15 ± 47.31 ms**	**+60.6% (−60.6% Δ)**
Attempted Reqs / Trial	69,223.0 ± 1,506.2	30,079.4 ± 532.4	Reconnect churn	68,056.8 ± 1,160.5	16,042.8 ± 5,284.6	Paused during sleep
HTTP 503s / Trial & Rej. Rate	0.0 (0.00%)	**17,126.8 ± 171.3**	**56.94% (56.95% avg*)**	0.0 (0.00%)	**128.0 ± 0.0 (exact!)**	**0.80% (0.86% avg*)**
Client Back-Off / Evaluation	n/a	n/a (ignores Retry-After)	**Unsuccessful**	0.00 ± 0.00 s	**511.91 ± 1.65 s / trial**	**Conditional Trade-Off**

Source: Dataset: historical_v4_20261019; Report Tables 5.3 & 5.4 (pp. 28–29). *56.94%/0.80% = ratio of means; 56.95%/0.86% = mean of trial ratios

Separate Verification-Host Rerun Results (21 Oct 2026)

Independent 80-trial paired replication on virtualized AMD EPYC 9V74 host, Port 18080 (Report Appendix B.7, Tables B.7–B.8)





Mode (fresh_rerun_20261021)	C	Baseline 2xx RPS	QLAAC 2xx RPS	Δ RPS	Baseline p95 / p99 (ms)	QLAAC p95 / p99 (ms)	p99 Red%	503 Total (5 tr) / Backoff
Standard (Port 18080)	2	37,065.64 ± 1,233.08	36,254.12 ± 997.66	−2.2%	0.100 / 0.202	0.102 / 0.202	+0.0%	0 (0.00%)
Standard (Port 18080)	8	24,702.80 ± 2,083.31	25,966.64 ± 973.23	+5.1%	0.460 / 7.766	0.418 / 7.390	+4.8%	0 (0.00%)
Standard (Port 18080)	32	26,315.90 ± 981.51	22,846.24 ± 867.64	−13.2%	0.418 / 52.716	17.640 / 21.944	**+58.4%**	**0 (0.00%, CAUTION)**
Standard (Port 18080)	64	26,205.88 ± 846.95	**4,662.02 ± 73.38**	**−82.2%**	0.414 / 113.210	2.658 / 90.162	+20.4%	**276,337 (54.24% shed)**
Retry-Aware (Port 18080)	2	36,594.26 ± 1,206.41	36,727.30 ± 1,786.18	+0.4%	0.098 / 0.200	0.100 / 0.198	+1.0%	0 (0.00%) / 0.00 s
Retry-Aware (Port 18080)	8	25,284.54 ± 559.12	25,267.98 ± 2,395.09	−0.1%	0.428 / 7.588	0.444 / 7.662	−1.0%	0 (0.00%) / 0.00 s
Retry-Aware (Port 18080)	32	26,100.96 ± 473.80	23,295.10 ± 569.28	−10.8%	0.414 / 52.868	17.370 / 21.476	**+59.4%**	**0 (0.00%) / 0.00 s**
Retry-Aware (Port 18080)	64	25,891.42 ± 667.90	**8,727.14 ± 167.96**	**−66.3%**	0.418 / 114.790	26.858 / 33.424	**+70.9%**	**640 (128/tr, 0.15%) / 452.48 s**

Source: Separate Dataset: fresh_rerun_20261021 (AMD EPYC 9V74, Port 18080, 5 × 10 s paired trials); Report Appendix B.7 Tables B.7–B.8 (pp. 51–53)

Analysis: p95 vs. p99 Rotation, Timelines & Failure Conditions

Why CAUTION raises p95 while cutting p99, how hysteresis shapes C = 64 timelines, and when QLAAC reduces performance

1. Why C = 32 Cuts p99 (−61.3%) While Raising p95 (~56×)

- **Baseline (50 reqs/conn):** Only 1 in 50 requests (2.0%) is the 1st request on a queued socket; 98% bypass the queue (~0.5 ms). Thus p95 stays at 1.31 ms while p99 absorbs 7 rounds of 200 reqs (201.93 ms).
- **QLAAC CAUTION (≤16 reqs/conn):** 1 in 16 requests (6.25% > 5%) now waits in the queue, shifting p95 into the waiting group! Approximate consistency model: $T_{wait} \approx 7 \times (4 \times 16 / X) \times 1000$ ms ≈ 74.8 ms at X = 5,988.46 RPS (v4 Std: p95 = 72.99 ms, p99 = 78.17 ms; Fresh Std: T_{wait} ≈ 19.6 ms vs p95 = 17.640 ms, p99 = 21.944 ms).

2. Diagnostic Timelines (Fig. 5.6) & Survivorship Bias

- **C = 32 & C = 64 Timelines (0.25 s Polls):** At C = 32, QLAAC enters CAUTION at t ≈ 0.7 s (q = 28/64, qw ~200 ms) and stays in CAUTION with 0 shed. At C = 64, q = 60/64 triggers OVERLOAD at t ≈ 0.95 s; 5-window recovery steps down to CAUTION at t ≈ 6 s and re-triggers OVERLOAD in 3 of 5 trials.
- **Cost of 503s & Survivorship Caveat:** At C = 64, Retry-Aware p99 (175.15 ms v4 / 33.424 ms Fresh) measures only admitted 2xx survivors while clients spend ~71–80% of trial time (511.91 s / 452.48 s) sleeping on Retry-After: 5.

3. Five Explicit Conditions Where QLAAC Reduces Performance or Fails (Report §5.5, p. 37)

- **(i) Low or Moderate Load (C = 2, 8):** When concurrency ≤ workers or queue depth stays below q = 0.20, QLAAC stays in NORMAL and provides no benefit over baseline.
- **(ii) Objective Is Maximum Throughput, p50, Mean, or p95:** At C = 32, QLAAC loses 13.7% / 13.4% throughput (v4) and raises p95 ~56-fold (1.31 → 72.99 ms); only p99 tail latency improves.
- **(iii) Non-Cooperative / Immediate-Reconnect Clients:** At C = 64 Standard mode, 503 shedding causes a reconnect storm (56.94% shed, −81.3% RPS) for only a 9.4% p99 gain.
- **(iv) Short Windows with Cooperative Retry-After Clients:** At C = 64 Retry-Aware mode, a 5 s Retry-After sleep consumes half of a 10 s trial (−76.6% RPS in v4, −66.3% in Fresh).
- **(v) When Low Trial-to-Trial Overload Variance Is Required:** At C = 64 Retry-Aware mode in v4, synchronized retry waves cause wide dispersion (RPS CV 33%; p95: 2.55–108.27 ms).

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 5.4–5.5, Table 5.6 & Figures 5.5–5.6 (pp. 32–37)

Limitations & Threats to Validity: Protocol & Testbed Scope

Why our server is a documented HTTP/1.1 subset and why benchmark claims are scoped to our loopback testbed

1. Protocol, Concurrency & Security Scope (Report §1.4, §5.5)

- **Documented HTTP/1.1 Subset (Not Full RFC 9112):** Supports GET, HEAD, POST, Content-Length (≤10 MiB), and keep-alive. Does NOT implement chunked request decoding (Transfer-Encoding: chunked → 501 Not Implemented, an explicit deviation from RFC 9112 §6.1/§7.1); Expect: 100-continue → 417.
- **Defensive Bare-LF Rejection:** Rejects bare LF line terminators with 400 Bad Request to prevent smuggling, even though RFC 9112 §2.2 permits recognizing bare LF.
- **Out-of-Scope Protocols:** No TLS/HTTPS, HTTP/2, HTTP/3, range requests, caching validators, content compression, or virtual hosting. Not production-ready.
- **Blocking Worker Pool & Slow-Client Risk:** 4 synchronous pthreads block per active connection (no epoll/io_uring); 4 slow clients trickling bytes within the 5 s deadline can monopolize all workers (Slowloris risk).
- **Controller Parameter Heuristics:** 1,000 ms sampling, 5-window recovery, and the 16-request cap were selected heuristically without a multi-parameter sensitivity sweep.

2. Experimental & Measurement Threats to Validity (Report §6.2)

- **Single-Host Loopback Contention:** Server and Python load generator run on the same host over 127.0.0.1, eliminating WAN RTT/packet loss while sharing host CPU and kernel scheduler.
- **Synthetic /health Micro-Workload:** Benchmarks exclusively test GET /health (89-byte in-memory JSON); disk-backed sendfile(2) static files and large POST /api/echo bodies were functionally tested but not load-benchmarked.
- **Closed-Loop Python Client Threads:** Python threading and GIL overhead can limit offered load at high concurrency compared to an open-loop C generator like wrk; at C ≤ 64, baseline queue depth never exceeded 60/64.
- **Short 10 s Window vs. 5 s Retry-After:** A 10-second trial window heavily penalizes 2xx throughput when Retry-Aware clients sleep for 5 seconds (50% of the trial duration).
- **Sample Size & Missing v4 Host Metadata:** 5 trials per cell without formal hypothesis tests; historical v4 CSVs did not log CPU model, clock speed, or temperature.

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 1.4 (p. 8), 5.5 (pp. 36–37) & 6.2 (p. 39)

Verification Chronology & Readiness-Check Validation Scope

Distinct 80-test vs. 92-test chronology, offline skip behavior, and post-benchmark readiness check (Appendix B.2 & B.8)

1. Strict Test Chronology: 80-Test Run vs. 92-Test Suite

- **Stage 1 — Earlier Run: 80 Tests (Appendix B.2):** 44 C17 unit tests (5 binaries: controller 13, parser 14, queue 4, responses 5, metrics 8) + 36 Python tests = 80/80 passing with server active (15 pass / 21 skip offline). C unit harnesses passed ASan+UBSan & TSan; gcc -fanalyzer 0 warnings.
- **Stage 2 — Readiness Suite: 92 Tests (App. B.8, Post-Rerun):** Added 12 readiness tests in tests/test_readiness_check.py. Server-active run (bash scripts/run_all_tests.sh): 44 C + 48 Python = 92/92 passing (0 skipped). Offline Python run without port-8080 server: 26 pass (14 accounting + 12 readiness), 22 server-dependent tests skipped.
- **No Post-Patch Benchmark Rerun:** The 80-trial fresh performance rerun (App. B.7) was completed BEFORE Stage 2; the performance sweep was NOT rerun after the readiness-check refinement.

2. Readiness Check (wait_for_server) & Timeout Scope

- **Application-Level Identity Probe:** Refined wait_for_server() in run_reproducible_evaluation.py sends GET /health and requires HTTP 200, valid JSON dict, status='healthy', server='HTTP-Server-C17/1.0.0', and protocol='HTTP/1.1' so an unrelated listener on port 8080 is never mistaken for our server.
- **Honest Timeout Qualification:** Uses time.monotonic() deadline and caps each HTTPConnection timeout to min(remaining, 1.0) and retry sleep to min(0.05, remaining). Bounds individual blocking calls/sleeps, NOT a hard preemptive cutoff against a slow-trickle peer.

Excerpt: benchmark/run_reproducible_evaluation.py (wait_for_server)

```
start = time.monotonic(); deadline = start + max_wait_sec
while (rem := deadline - time.monotonic()) > 0:
    conn = HTTPConnection(host, port, timeout=min(rem, 1.0))
    conn.request("GET", "/health", headers={"Host": hdr})
    resp = conn.getresponse()
    if resp.status == 200:
        d = json.loads(resp.read(65536).decode("utf-8"))
        if d.get("status") == "healthy" and d.get("server") == ID:
            return (d.get("protocol") == "HTTP/1.1")
    time.sleep(min(0.05, max(0.0, deadline - time.monotonic())))
```

Test Suite Component	Stage 1 (App. B.2)	Stage 2 Active	Stage 2 Offline	Scope Verified
5 C17 Unit Binaries (make test)	44 pass	44 pass	44 pass	Controller (13), Parser (14), Queue (4), Resp (5), Metrics (8)
Python Accounting / Protocol / Sec	36 pass	36 pass	14 pass / 22 skip	22 live-server tests skip via @skipUnless if port 8080 offline
tests/test_readiness_check.py	(Post-B.7)	12 pass	12 pass	Ephemeral-port mock servers (Cases 1–6) + live C17 binary (Case 7)
Total Automated Tests	**80 pass**	**92 / 92 pass**	26 Py pass / 22 skip	Old 72-test claim & unverified 06_test_results.png excluded

Source: CN_Project_Final_Report_Final_Submission.pdf, Section 4.4 (p. 25), Appendix B.2 (pp. 46–47) & Appendix B.8 (p. 54)

Conclusion, Quantified Findings & Future Work Proposals

Summary of delivered C17 system, qualified experimental findings (Report §6.1), and unimplemented future proposals (§6.3)

1. Engineering Contribution & Quantified Findings (Report §6.1)

- **From-Scratch C17 HTTP/1.1 Server:** Implemented on raw POSIX TCP sockets and pthreads (3,569 snapshot / 3,573 ZIP lines in 12 .c files, 1,144 lines in 11 .h files), verified by 92 automated tests (44 C + 48 Python).
- **Low & Medium Load (C = 2, 8 — NORMAL):** Zero 503 rejections and comparable performance to baseline (−4.3% to +1.7% RPS in v4; −2.2% to +5.1% in Fresh Rerun).
- **Medium-High Load (C = 32 — CAUTION Fair Sharing):** 16-request keep-alive rotation cuts p99 tail latency by 61.3% in both v4 modes (201.93 → 78.17 ms Std; 206.97 → 80.15 ms Retry) and 58.4%–59.4% in Fresh Rerun with ZERO 503 rejections, trading 10.8%–13.7% RPS and raising p95 (~1.3 → 73–75 ms v4).
- **High Load (C = 64 — OVERLOAD Client Divergence):** Fails under immediate-reconnect Standard clients (56.94% shed, −81.3% RPS, +9.4% p99 in v4). Under cooperative Retry-Aware clients (128 503s/trial, 0.80% shed, 511.91 s sleep), cuts admitted p99 by 60.6% (444.17 → 175.15 ms; +70.9% in Fresh) at a 76.6% (66.3% Fresh) throughput cost.
- **Overall Takeaway:** QLAAC is a conditional, regime- and client-dependent latency–throughput trade-off, not a universal improvement.

2. Future Enhancements — Proposals Only (Report §6.3)

- **1. Leading Queue-Age Telemetry Signal:** Proposal: Inspect the age of the oldest connection currently waiting at queue->head (monotonic_ns() − head.enqueued_ns) on each sampler tick so QLAAC reacts before worker dequeue.
- **2. Continuous Keep-Alive Cap & Sensitivity Study:** Proposal: Systematically sweep keep-alive caps (8, 16, 32) and thresholds, and scale max_requests continuously with queue occupancy K(q) to smooth the C = 32 p95 shift.
- **3. Open-Loop & Multi-Minute Benchmarks:** Proposal: Evaluate with a C open-loop generator (e.g., wrk) over 60–120 s windows with jittered Retry-After client backoff to measure steady-state recovery.
- **4. Hardware Telemetry & Diverse Workloads:** Proposal: Log CPU clock frequency and thermal sensors per trial; benchmark static file delivery (sendfile) and large POST /api/echo payloads under load.
- **5. Extended Protocol & Event-Driven I/O:** Proposal: Add RFC 9112 chunked request decoding, range requests, TLS, and hybrid epoll/io_uring I/O to mitigate Slowloris worker starvation.

Source: CN_Project_Final_Report_Final_Submission.pdf, Sections 6.1 & 6.3 (pp. 38–39) — Proposals clearly distinguished from completed work

IEEE References [1]–[11]

Normative IETF standards, server concurrency architectures, and queue-delay management literature (Report Section 7, p. 40)

Normative RFCs & Server Architectures [1]–[6]

[1] R. Fielding, M. Nottingham, and J. Reschke, Eds.: "HTTP/1.1," Internet Engineering Task Force (IETF), RFC 9112 (STD 99), Jun. 2022.

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds.: "HTTP Semantics," Internet Engineering Task Force (IETF), RFC 9110 (STD 97), Jun. 2022.

[3] W. Eddy, Ed.: "Transmission Control Protocol (TCP)," Internet Engineering Task Force (IETF), RFC 9293 (STD 7), Aug. 2022.

[4] M. Welsh, D. Culler, and E. Brewer: "SEDA: An architecture for well-conditioned, scalable Internet services," in Proc. 18th ACM Symp. Operating Systems Principles (SOSP), Banff, AB, Canada, 2001, pp. 230–243.

[5] M. Welsh and D. Culler: "Adaptive overload control for busy Internet servers," in Proc. 4th USENIX Symp. Internet Technologies and Systems (USITS), Seattle, WA, USA, Mar. 2003, pp. 43–57.

[6] B. Schroeder and M. Harchol-Balter: "Web servers under overload: How scheduling can help," ACM Trans. Internet Technol., vol. 6, no. 1, pp. 20–52, Feb. 2006.

Performance, Queue Delay & Tail Latency [7]–[11]

[7] D. Pariag, T. Brecht, A. Harji, P. Buhr, A. Shukla, and D. R. Cheriton: "Comparing the performance of web server architectures," in Proc. 2nd ACM SIGOPS/EuroSys European Conf. Computer Systems (EuroSys), Lisbon, Portugal, 2007, pp. 231–243.

[8] K. Nichols and V. Jacobson: "Controlling queue delay," ACM Queue, vol. 10, no. 5, pp. 20–34, May 2012.

[9] J. Gettys and K. Nichols: "Bufferbloat: Dark buffers in the Internet," ACM Queue, vol. 9, no. 11, pp. 40–54, Nov. 2011.

[10] J. Dean and L. A. Barroso: "The tail at scale," Commun. ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[11] J. D. C. Little: "A proof for the queuing formula: L = λW," Oper. Res., vol. 9, no. 3, pp. 383–387, Jun. 1961.

Thank You · Ready for Live Two-Terminal Demonstration, Parameter Change & Viva Voce — Yash Pradhan (24BCE0702) & Anjini Pandey (24BCE0714)